

NittanyAuction

Released 01/31/2026

NittanyAuction is a startup project which aims to provide members of Lion State University (LSU) to sell and buy used or unused goods via online auctions. Hilbert Dude, the founder of this startup, envisages an undeveloped market for such an e-commerce platform in university campuses around the world. After conducting a preliminary market investigation, Mr. Dude realized the value and potential of such a platform, which is not only beneficial to the members of a university but also to the local economy in the campus and its surrounding community. He came up a simple yet cool brand name “**NittanyAuction**” and managed to convince some rich dudes (in the Dude Family) to sponsor an initial round of funding. Now the NittanyAuction project is ready to fly.

Mr. Dude is not a technical person, so he decides to find a technical team to prototype and validate his business ideas. Through the connection of Lions Alliance (a cross-university organization), he reached out to Professor Lee at Penn State University (PSU), home of Nittany Lions, for assistance to conduct a feasibility study on the ideas he has in mind. After discussing some requirements and manpower needed, Mr. Dude entrusted Professor Lee and his teaching team to advise and support students in CMPSC431W to systematically design and implement prototypes of **NittanyAuction** to validate his ideas. The goal is not to produce a complete implementation of the system, as it would require too much time and resources for the students. Instead, students (organized in teams) will focus on i) the process of the database design that may pose a significant risk due to uncertainty and the lack of understanding in requirements, and ii) the prototyping of certain system functionality as a proof of concept. It is anticipated that based on careful examination of the tested prototypes, valuable insights and lessons may be obtained. Widely accepted, this design-prototype approach of feasibility study is often adopted when the requirements are not well understood. Moreover, a successful prototype can potentially serve as the foundation for the future production ready system. Furthermore, an impressive demonstration of the prototyped system, functions, and unique features will surely demonstrate the strong creative and technical ability of PSU students, reaffirming the strong reputation in the IT industry.

Mr. Dude and Professor Lee ask you (student teams in CMPSC431W) to envision what **NittanyAuction** will be. Mr. Dude has tried to communicate this project roughly and abstractly and expect students to figure out and fill in all the missing details. Based on the provided project description, (each student team in CMPSC431W) will design a database-backed web application to buy/sell products on campus by members of the university and local community. You need to analyze the requirements of **NittanyAuction** to determine the system’s functionality and to identify the data needed for the system’s functionality. Furthermore, business rules (integrity constraints) need to be determined and imposed upon the needed data. Accordingly, you will go through the process of database design and then implement a prototype to demonstrate system functions and database design. As parts of the prototype, you will need to implement programs to access data from the designed database to support the functionality of the system.

The project is divided into two phases:

Phase 1: Requirement Analysis and Database Design

During this phase, student teams focus on understanding the core requirements of the platform and translating these needs into a comprehensive database design. Key tasks in Phase 1 include:

1. Requirement Analysis: Identify the essential functionalities of **NittanyAuction**, such as user login, user registration, auction listings, auction bidding, etc. as well as the business rules/policies to be followed in the platform. The identified system functionalities should be described in detail and illustrated with user interface mockups to explain how they work.
2. Conceptual Database Design: Based on the requirement analysis, use the entity-relationship (ER) model to express the information needs and business rules (i.e., data and integrity constraints) required to support the platform's functions.
3. Technology Survey: Research current web and database application technologies, including web frameworks, programming languages, development tools, and database management systems. Student teams will compare these technologies to the recommended tools: Flask (Web Framework), Python (Programming Language), PyCharm (IDE), and SQLite (Database).
4. Logical Database Design and Normalization: Create a normalized schema to ensure data efficiency and avoid redundancy.

By the end of Phase 1, student teams need to produce a detailed, well-documented report outlining their analysis, findings, designs, and technology recommendations.

Phase 2: System Implementation and Functional Prototype

In the second phase, student teams will bring their designs to life by implementing the core functionalities of **NittanyAuction**. Tasks include:

1. Database Population: Filling the database with sample data to demonstrate the platform's capabilities.
2. System Functionality: Developing features such as user login, user registration, auction listings, auction bidding, and so on.
3. User Interface Design: Creating an intuitive and visually appealing interface to ensure ease of use for both businesses and suppliers.

OVERALL REQUIREMENTS

Before we jump into the description of each phase, please note what you (student teams in CMPSC431W) are expected to fulfill while working on the project.

Team Project

This is a **TEAM Project**. The **design and implementation must be completed collaboratively** by the members of each team, based on the team's **shared ideas and joint decisions**. The

success of project heavily relies on collaborative team effort and individual commitments. All the teams should manage your project's progress and involve all team members in the teamwork. Individual and team effort (activities, assignments, etc.) should be documented in progress reports (at least bi-weekly), and included in the Appendix of the final report. You are encouraged to use a task management tool. Make sure you carefully plan each task in the project and keep track of the conversation/discussion between teammates. Information on task management tools as well as GitHub can be exported and incorporated into the progress reports (and the final report). While each team will receive one grade, we will use peer reviews of the team to adjust the individual grades at the end.

Document Report and Formatting

The project report should be written in your own words. Reusing (copying) parts or all of the project description directly not only receives no credit but also violate academic integrity in terms of Plagiarism!

You can prepare your documents using any document typesetting languages/tools. We will provide a document template (as .docx file) that you are more than welcome to directly follow. However, you are encouraged to create a template of your own, as long as it maintains a professional image for your cover page and includes all the components from our provided template. Your report should be converted into one PDF file for submission **electronically** on Canvas.

Project Management

The success of this project heavily relies on your own personal effort and commitment. As denoted on the Syllabus and Course Schedule, the project is segmented into two phases. Thus, there will be no intermediate submissions except for a progress review before the final demo. Students are expected to set goals and maintain individual milestones, documented as part of the Phase 1 report. For those who do not have prior background on web programming, it is highly recommended to start reading documentation and tutorials and practice them during Phase I.

Since there are numerous web application tools and frameworks, it is impossible for the teaching team to provide guidance on every combination of tools. If students choose to use the recommended platform and tools (i.e., *Flask Web Framework*, *Python*, *PyCharm Professional IDE*, and *SQLite*), the teaching staff are more than happy to provide guidance and assistance. On the other hand, you may choose to adopt platforms, tools, programming languages and database management systems of your preference, but you are on your own in this case.

PHASE I – DATABASE DESIGN AND TECHNOLOGY SURVEY

Checklist

Tasks
Requirement Analysis
Conceptual Database Design
Technology Survey
Logical Database Design and Schema Normalization

Task 1: Requirement Analysis

NittanyAuction aims to become an auction-based online e-commerce platform where university members and local business can buy and sell goods online. Thus, it may help to explore some online e-commerce websites such as eBay, Amazon, Walmart, Facebook market, etc. for reference to better understand the expected (and alternative) functions and requirements for *NittanyAuction*. The following is a general description of the expected functionality for the project, reflecting Mr. Dude's non-technical point of view of the system. In this task, you should envisage what the system should look like and how it may operate to enable the campus auction business. You are required to analyze and describe in detail the system requirements, including the system functions/operations, data needed to support those system functions/operations, the business rules (integrity constraints) of *NittanyAuction* to be imposed on the data, and how/where they will be stored and accessed to support the system functionality. Please note that in this task, you are expected to elaborate upon the system functionality in your own words with figures and identify (write down) the data needed to support each functionality as well as the integrity constraints to be imposed on the data. We first describe *NittanyAuction* from the users' perspective, then add more information on the system.

1. **Registered Users:** To operate on *NittanyAuction*, a user needs to be registered in the system. A registered user is identified by a user ID (e.g., email address) and authenticated with a password. There are three different types of users who will use the system: Bidders, Sellers, and HelpDesk Staffs (HelpDesk in short). In the following, we describe what those three types of users are expected to do with the *NittanyAuction* system in their respective roles.
 - a. **Bidders:** While the bidders are targeting on all members of LSU (i.e., students, faculty members, staffs, and so on), we will consider only students in this initial effort. Therefore, all LSU students are bidders by default. It is important to collect and maintain detailed information about bidders in *NittanyAuction*, including e-mail address, name, address (which consists of street, city, state, and zipcode), phone number, credit card info (including type of credit card, credit card number, expiration date), major, age, annual income, and etc. Among them, the last four items are collected for marketing analysis, but we won't tell the bidders that 😊.

A bidder may log into **NittanyAuction** with their email address (which serves as the user ID) and their existing password. After logging in, they can update their personal information (except for their ID) and reset login password. A bidder can also manage payment information, add/remove credit card number which is only viewable/accessible to herself/himself.

Additionally, a bidder may browse the product listings via a category hierarchy (subcategories will be selectable in a list/menu) to find an interested product and then start bidding the products. He/She may also send questions about a particular product listing to its sellers. In **NittanyAuction**, products are organized based on a category hierarchy. For each product listed, seller info and related product details are shown to the bidder. Products which are already sold (auction successful) or terminated by the seller (auction failed) should not be viewable/listed to buyers.

After a bidder wins an auction, he/she pays the winning price right away. After the transaction is completed, the listing will be archived and no longer listed. The bidder will be able to rate the seller after the transaction completed. Note that ratings, meant to serve the community regarding the reputation of sellers, is based on the 5-star rating system.

Some bidders may want to put their own items in auctions as well. To do so, the buyer need to fill out an application form and submit it to the HelpDesk for approval.

- b. **Sellers:** Similar to bidders, a seller also needs to log into **NittanyAuction** with her/his user ID (i.e., email address) and password. After the seller log in, she/he can update some account information and reset their login password. Sellers are either students of Lion State University or approved local business vendors. Different from bidders, bank routing number, bank account number, and a balance in **NittanyAuction** are maintained for sellers. Additionally, for approved vendors, information such as business name, business address, and customer service phone number must be maintained. Note that **NittanyAuction** provides an opportunity for the local vendors to market their special products, which are unique and different from big brand-name products. Thus, these products will only be made available by local vendors for auctions in some promotional occasions. If a local business seller decides to leave the market, the information of its products will be automatically removed and no longer available in the market.

Sellers need to enlist/publish the products they want to put up for auctions on **NittanyAuction** by providing the product information, the category of product, a *reserve price* (which is hidden from the bidders) and an auction stop time. A reserve price is the minimum price a seller is willing to accept for the item. At the end of the auction, if no bid is higher than the reserve price, the seller is not obligated to sell the item.

For a listed product, a seller can view all the information for the product, and the questions asked by bidders. Sellers can see the ratings on themselves given by the bidders. Sellers can list products under existing categories for sale. If some new products cannot find a fitting category in the category hierarchy, the seller needs to submit a request to HelpDesk.

Once approved, the HelpDesk Staff will create a new category in the category hierarchy as appropriate.

When an auction is ended (either successful or fail), the product will be delisted (no more visible to the bidders) but the auction information including the bids will be archived in the system. For a successful auction, once the payment is completed by the bidder, the seller will receive the payment in the account.

Intuitively, every product needs to have exactly one category, and every product item is listed for auction by only one seller. For example, John and Jane have a shared textbook. They cannot create two separate lists for the same book. While sellers usually are not limited to selling only one product at a time, we assume a product list contains only one product item for simplicity. Except the aforementioned case where information about products made by local business vendors information got deleted automatically when they leave the market, *NittanyAuction* is responsible for maintaining historical listings of sold products and bidding history for market research and improvement purposes, regardless if their sellers are no longer listed in the platform.

- c. **HelpDesk:** *NittanyAuction* will hire a team of IT staffs to manage the system and provide various services. For example, other users (bidders and sellers) may not change their user IDs. They need to submit a request to the HelpDesk team to make the change. Likewise, for a bidder to upgrade his role to a seller and for a seller to add a new category in the category hierarchy, these requests all need to go through HelpDesk. Additionally, they also perform some marketing analysis tasks, e.g., listing the number of auctions put forth by sellers in different majors.

The above descriptions are given by taking the views of different users of *NittanyAuction*. In the following, we describe the Products, Bids, Categories, which are important information to be managed in *NittanyAuction*.

- d. **Products:** Every product belongs to exactly one category. Every product item is listed by exactly one seller. For each auction, only one product is listed. If multiple items of the same product, e.g., two cow books, are listed, they are auctioned together. Products should contain some useful information that help the buyer to find the product (i.e., via browsing or search) before making a bid. Important information regarding a product like title, product detail, category and reserve price are required. However, additional information which you (CMPSC431W Student teams) believe to be important can be included, as creativity is always highly appreciated and encouraged! Finally, as mentioned previously, products by local business are unique. Their information will no longer be available if the business vendor leaves the market.
- e. **Bidding:** A bidder can place one or more bids on a product list as long as they are not the seller of the product. Each bid that is placed must be at least \$1 higher than all previous bids and must be placed before the auction ends. At the end of the auction, the seller via

NittanyAuction notifies all bidders the highest bid. With the notification, the bidder who won the auction can complete the payment transaction. An auction starts when the seller publishes the product for auction and ends exactly at the time the seller specified. Again, all historic auction information should be maintained no matter the transaction is successful or failed.

- f. **Category hierarchy:** A category is a generalization of a group of products. It is a higher-level unit of products. Categories available in the platform are predefined by the HelpDesk team. As mentioned, for sellers to put up a new product of some non-existing category for auction, they can send a request to HelpDesk for approval. Once approved, it's the job of HelpDesk to create a new category for the item in the category hierarchy. Note that, in the category hierarchy, one category may have multiple subcategories, except the leaves (i.e., the lowest level) of the category hierarchy. The number of products within a category is unlimited. Therefore, the buyers can traverse the category hierarchy to narrow down the interested products in a specific category.

From the above description, you may tell that Mr. Dude is not a CS engineer. The description mainly reflects some rough ideas he had in his mind. For example, he didn't even think about how a bidder may *search* available products, in addition to browse the category hierarchy. Moreover, after finding some interesting products, a bidder may also see what other products are offered by the same seller. In addition, there are various potential ideas to explore. To get the project moving forward (and to educate him, your customer), you may fill in the missing information, make suggestions, and inject your own ideas with clear elaboration and justification in your Phase 1 report.

Additionally, to impress Mr. Dude with insights and strengths of your team, please extend NittanyAuction with at least one new features/functionality not described above, along with new data needed and business rules identified. Note that the first new feature is required and additional new features may earn you extra credits.

Task 2: Conceptual Database Design

Based on the result of Task 1, you should present an *entity-relationship (ER) diagram* describing your conceptual database design. Also, your report should include a narrative description on all aspects of the diagram in detail. In addition to an overall ER-diagram of your conceptual design, parts of the ER diagram such as entities, relationships and integrity constraints are expected to be explained in detail. Note that the conceptual database design is application-oriented. Please do not assume the underlying database system (just yet) and thus do not map your design into relations in this task.

Task 3: Technology Survey

An aspect of working in the IT industry is to always keep your knowledge updated with the current market trends and technology innovations. Thus, for the project, it's important for you to have a good knowledge of the current web programming and database technology. A technology survey

is meant for you to research a breadth of various web programming frameworks (for application development), programming languages, tools (e.g., IDE), and database management systems as well as the trends in those technological areas. Beside taking into consideration the web programming stack suggested by the CMPSC431W teaching team, i.e., Flask, Python, PyCharm IDE, SQLite, you have to come up with **at least three** trending alternatives in to make a comparison with the software/language mentioned above. It is expected for you to write **persuasively** about which tools you think as the best fit for the project. Please consider as many aspects as possible regarding the roles of those technologies in your project, and to **justify your own recommendation by discussing the reasons, pros and cons of your choices.** Also discuss the impact and relevance of those technology trends to the computer science fields and a broader segment of society or business/industry.

Task 4: Logical Database Design and Normalization

Based on the Requirement Analysis and Conceptual Database (ER) design you developed in Task 1-2; you will finalize the relational schema for the database in *NittanyAuction*. You should produce a refined schema that reduces data redundancy to an acceptable level (i.e., the final schemas need to be at least in the 3rd Normal Form) while not unduly affecting performance. Your schemas should also support the enforcement of most, if not all, of the integrity constraints that you identify in this phase as well as those newly identified or added after relational schemas are generated from the ER diagram. **In this task, we assume two functional dependencies have been identified: a) zip codes in address determines state and cities; b) student email address which determines the login password.** You should present the specific details of how the process of schema generation and normalization is done, i.e., you are expected to apply both of translating the ER diagram to relations and then perform schema normalization. Your report should also include a narrative description with illustration of the translation (mapping), normalization and SQL statements

Grading

Different from exams, the project provides a way for students to show their commitment and effort in learning and practicing the knowledge learned from CMPSC431W. While the course project is work intensive, student teams which show their strong motivation and effort through the high quality of their reports will be rewarded. The project report, fulfilling the Writing requirement of the Penn State curriculum, is graded in terms of correctness, completeness, presentation, and clarity. Again, your effort will be reflected in your report and rewarded.

Again, please be reminded that the project is work intensive -- start early and prioritize your time.

As mentioned earlier, **1 new functionality (i.e., proposed by your team, not stated in the above project description) is required**, which should be expressed explicitly and clearly within the report. The data needed for the new functionality should also be specified, modeled, and refined in Task 1, 2, and 4, like other data in the project. This is graded on novelty of the new functionality and clarity your team presents in various tasks of Phase 1. In addition to the required new functionality, extra credit of up to 10% will be rewarded for reports which include additional new features/functionality (please mark it explicitly!)

Submission

As mentioned above, a template will be provided as a .docx which serves as a mandatory style guideline. **The deliverable is a report in PDF containing the requirement analysis, conceptual database design, technology survey, and logical database design and normalization.** The document must have page numbers, section numbers and a table of contents. Figures should be used for illustration of your design. In addition, the document should also include a project plan (including schedule, deliverables, and milestones) as an appendix in the project report.

The length of the body of this report, excluding the cover page, table of contents, list of figures and list of tables, is expected to be **no less than 30 pages**. However, based on our experience, 30 pages are usually not enough to complete all the tasks in Phase 1 of the project with sufficient detail! There is no upper limit on the number of pages in your document. You may use as many pages as you like to produce a complete and impressive report.

Turnitin will be where you submit a .PDF version of your report. As stated in classes as well as the syllabus, academic integrity is the most important and there will be no tolerance and exceptions towards violation. An incomplete but attempted submission is always far better than committing an academic integrity violation.

PHASE II – SYSTEM PROTOTYPE

Phase 2 Released 03/09/2026

The PSU students have sent in their design proposals to Mr. Dude, who with the assistance of Professor Lee and his teaching team has determined the best conceptual and logical database designs which match what they had in mind. Now, Mr. Dude wants to validate the idea by prototyping a web application based on the selected logical database design and see how it work. To make it easier to assess different implementations, Mr. Dude has compiled a dataset (in the form of CSV files) based on a reduced set of relational schemas, which will be used by the students to populate the database and test the prototyped system functionality. For evaluation, **the student implementations will be tested/evaluated based on the ability to execute the designated tasks and the overall user experience (UX) from using the website.**

The relational schemas for *NittanyAuction* have been provided in a separate document. You are expected to follow these relational schemas for Phase 2. However, you can make changes to the schemas as long as you use the data from the CSV files we provide. You may also extend the schemas to create new tables for new features you would like to implement (upon approval). In that case, you will need to prepare data for the corresponding tables for testing and final demonstration.

Checklist

✓	Tasks
	Database Population
	Functionality Implementation
	Reflections and Video Presentation
	Final Deliverables

Project Description

In Phase II, you will be provided with the following materials:

- i) a database schema design, and
- ii) CSV files which contain the raw data (released on Canvas under /Modules/Project/). You are supposed to populate the data in accordance with the schema into the database. In addition to this, you will implement the web application for *NittanyAuction* and demonstrate the functionality of *NittanyAuction* as detailed in Task 2 below.

Task 1: Database Population

The first task is to create and populate the database. You will be provided with large data files where the columns contain values for various fields. However, please note that those are RAW

data, i.e., they may not comply to the integrity constraints identified in Phase 1. You are supposed to organize the raw values in the files based on the schema provided to create all the necessary tables, define primary keys, foreign keys, and specify any integrity constraints as appropriate. In addition, define any views of the schema as you see appropriate. Finally, it's important for you to inspect all the columns and the corresponding data types in the CSV files before you proceed to populate the database.

To support additional functionality you wish to implement, you are encouraged to augment the schema and add your own data as needed. You can use Python libraries (e.g., Pandas) or other database facilities/tools to parse the CSV files and insert values to the table. You should provide (and maintain) the scripts/codes (or documentation of the steps) used for your data population, so that you can easily regenerate the database or restore the database states. In the past CMPSC431W courses, there were some cases of data loss or tables corruption during implementation. Such data loss/corruption will not be used as an excuse for project incompleteness or deadline extension. Therefore, it is crucial to be well-prepared for potential issues by maintaining your scripts, code, and documentation for database population and regularly backing up your project files.

Task 2: Functionality Implementation

You need to prototype (i.e., implement) the system functionality as specified below and demonstrate how they operate on your final system prototype. The following is a description of the expected functionality.

1. **User Login:** This is for a user (seller, bidders or helpdesk) to log in. Note that the log-in information of users is stored inside the *NittanyAuction* system (after data population). The system recognizes the user by their email and password. For security reasons, passwords must be *hashed* when stored and must not be visible while being entered.

Upon successful login, users will be directed to a role-specific welcome webpage. For example:

- **Bidders** may view and edit personal information, browse the categories/products, manage shopping carts, and place orders.
- **Sellers** may view their published products, edit product details, and review buyer feedback.
- **HelpDesk staff** may access pages that display pending requests, user management, or system administration tasks.

Please note that these descriptions of role-specific welcome webpages are provided only as examples. The primary focus of this functionality is to implement a secure and functional user login process. Your team can design customized pages to support user roles according to your own vision and user experience (UX) considerations.

2. **Category Hierarchy:** All products available on *NittanyAuction* are organized using a predefined category hierarchy. This hierarchy consists of nodes, each representing a category with a descriptive name. Every category node is associated with a set of products belonging to that category, and may also include several subcategories, each containing their own specific products.

The root node of the hierarchy, labeled as '**All**', does not directly contain any products. Instead, it serves as the starting point for browsing and contains multiple top-level subcategories. When a user clicks on a subcategory (to traverse the hierarchy), the system dynamically queries the database to display the next-level subcategories and the associated products. Clicking on an individual product name should lead the user to a detailed page displaying information such as price, seller details, reviews, and ratings.

Important Implementation Note:

The category hierarchy is logically existing in the database. Again, your system should dynamically retrieve categories and subcategories via database queries while traversing the hierarchy. Hardcoding or statically retrieving the entire category structure at once in the application code is not recommended and will receive only partial credit.

3. **Auction Listing Management (Seller):** This functionality enables a **seller** to create, publish, manage, and remove an auction listing on **NittanyAuction**. Before a listing can be published, the seller must enter all required listing information (e.g., title, item details/condition, category, reserve price, the auction limit parameter **Max_bids**, and so on). Once published, the listing must become **immediately visible** under its category when users browse the category hierarchy.

Sellers must be able to view **all of their listings**, clearly grouped by status:

- **Active:** published and available for bidding
- **Inactive:** removed by the seller before being sold
- **Sold:** auction ended successfully and no longer available to browse/bid

Sellers may edit listing details **only when the listing is not sold**, and the system must enforce the following rule:

- **Active listing update restriction:** If bidding has started on an active listing (i.e., at least one bid has already been placed), the listing **cannot be updated**. The UI should show a clear message explaining why the update is blocked.

Sellers may also **take an active listing off the market** (change it to inactive). When a listing is taken off the market, the system must **preserve historical/administrative information** about this removal, including:

- the **remaining number of bids** at the time of removal, and
- the **reason** the seller provided for removing the listing.

Schema extension requirement: The provided schemas/datasets may not include “remaining bids” or “removal reason,” so teams may need to **extend the database** (e.g., new attributes or a separate audit table) to support recording this information for newly created listings.

4. **AuctionBidding (Bidder):** This functionality enables a **bidder** to place bids on an active auction listing. A bid must satisfy all of the following rules:
 1. **Increment rule:** Each new bid must be at least **\$1 higher** than the current highest bid.
 2. **Auction end rule:** Bids must be placed before the auction ends. For this prototype, the auction ends when the number of bids reaches **Max_bids** (instead of using a stop time).

3. **Turn-taking rule:** A bidder **cannot place consecutive bids** on the same listing; they may bid again only after another bidder has placed a competing bid. After a bid is placed, the system must display the auction detail page again, including the **updated remaining number of bids** and the updated highest bid. The UI should clearly indicate whether the bidder's bid was accepted or rejected and why (e.g., "bid too low," "auction ended," "you must wait for another bidder").

Auction completion and notifications: When the auction ends (i.e., Max_bids is reached), the system must **notify all bidders** of the result.

- If the final highest bid is **greater than or equal to the reserve price**, the highest bidder is the winner and should be guided into the **payment process**.
- Otherwise, the auction is unsuccessful and must no longer be available for bidding or browsing.

Payment flow: The winning notification must link the winning bidder to a payment page where credit card information can be entered or retrieved. After submission:

- the payment transaction must be recorded in the database, and
- the auction listing status must be changed to **"Sold"** and kept in the database for history, but **must no longer appear anywhere in NittanyAuction browsing/search results**.

5. **UserRegistration:** This functionality enables new users to **register an account** on *NittanyAuction*. New users must specify their intended **role** upon registration, choosing among Bidder, Seller, or HelpDesk staff (note: HelpDesk accounts might require additional administrative approval).

For registration, users are required to provide essential personal information (which can be found in relevant user tables) to ensure secure authentication and to facilitate system operations. Note that account creation for HelpDesk staff is typically handled internally. No external registration is permitted unless authorized by system administrators.

Once all necessary details are submitted and verified, the new user account is activated, and the user can immediately access the relevant platform features based on their designated role.

6. **UserProfileUpdate:** This functionality allows existing users to update their personal profile information. After logging in, users should be able to review, modify, and save changes to their account details, ensuring their information stays accurate and current. The following rules must be enforced for this functionality:
- **User ID (i.e., email address)** cannot be modified by the user directly. If a user needs to change their registered email address, a formal request must be submitted and approved by the HelpDesk team.
 - **Password** (If updated, the new password must be securely **hashed** for storage and must not be displayed visibly during entry.)
7. **Product Search:** This functionality enables bidders to efficiently search for products on NittanyAuction. Bidders can search for products by entering specific keywords and

specifying conditions such as a price range. The system should accept keywords matching attributes such as product title, description, category, or seller name.

Upon submitting a search, the system must dynamically query the database and clearly display a list of products matching the specified search criteria. Each listed product should display relevant details, including the product name, price, seller information, and availability status, facilitating intuitive user experience.

Important Implementation Note:

You are **not permitted** to use external search libraries or APIs for this functionality. The search feature must be implemented through direct and dynamic queries to the database.

8. **Rating:** After a bidder **wins an auction and completes payment**, the bidder may submit a **seller rating** (e.g., 1–5). The rating must be stored and used to compute/update the seller's **average rating**, which must be displayed wherever seller information is shown (e.g., seller profile, listing detail page).

Implementation notes:

- Only bidders with a **successful, paid** auction should be allowed to rate the seller for that auction.
- The system should prevent duplicate ratings for the same completed auction transaction.

In addition to the functionality listed above, you may choose to implement one or more of the functionalities below for extra credits.

Optional functionality to implement:

1. **HelpDesk Support:** This functionality allows buyers and sellers to request support from the **HelpDesk** team through a dedicated user interface.

Specifically, buyers and sellers can submit various types of administrative or support requests, including:

- Changing user IDs
- Adding new product categories
- Performing market analysis

Once a request is submitted, it is initially assigned to a general account (helpdeskteam@lsu.edu) as "**unassigned**." All HelpDesk staff members should have a clear view of incoming unassigned requests, as well as requests already assigned to themselves. HelpDesk staff members can:

- Claim an unassigned request (originally assigned to helpdeskteam@lsu.edu).
- View and manage their assigned requests.
- Mark requests as "**completed**" once the required actions are performed.

Important Implementation Note:

Due to limited time for final demonstration, your team only needs to implement the

functionality related to the request of **"adding a new category"**. Which include receiving requests, claim unassigned requests, view and manage their assigned requests. After processing this request, the HelpDesk staff must clearly update the request status to **"completed"**, reflecting this status change accurately in the system.

2. **AuctionPromotion (Seller):** This functionality enables a **seller** to promote an auction listing by paying a promotion fee equal to **5% of the reserve price**. When an auction is promoted:
 - it must appear at the top of the category hierarchy's **"All"** page, and
 - it must also appear when its associated category page is visited.The system must record the promotion action in the database (e.g., promoted flag/status, promotion timestamp, fee paid). Promoted auctions must still obey all normal auction rules (bidding, ending, sold/removed behavior).
3. **Shopping Cart / Watchlist (Bidder):** This functionality allows users to add auction listings they are interested in to a **Shopping Cart / Watchlist** for convenient access later. The cart should:
 - clearly list saved items, including listing title, current highest bid (or current price), remaining bids, seller information, and listing status;
 - allow users to add/remove items;
 - allow bidders to place a bid directly from the Shopping Cart, if the listing is **Active**
 - persist across logouts/logins (i.e., saved in the database, not only in session memory).

Important Note on Web Interface Design

While the correctness of the functionalities outlined above is essential, creating an **intuitive, visually appealing, and user-friendly web interface** is equally important to the overall success and effectiveness of your system prototype. A well-designed interface significantly enhances user experience (UX), making interactions smooth, logical, and enjoyable for bidders, sellers, and HelpDesk staff.

Thus, your team is strongly encouraged to dedicate attention and effort to designing an attractive and functional web interface. Prioritize clarity, ease of navigation, and visual coherence in your design decisions, as this will greatly influence the effectiveness and impact of your final demonstration.

Remember that the quality and usability of your web interface can significantly impress both the teaching team and Mr. Dude, effectively showcasing the strength of your design, implementation, and technical capabilities.

Finally, risk management is critical to your project. To protect your implementation effort and product from disasters, please backup all the data, codes, scripts, and documentation in safe external storage.

Task 3: Group Video Representation - Reflection

At the end of project, your team will revisit the decisions made throughout different stages of the project. It's beneficial for your team to flash back what you expected at the beginning of the semester regarding the project and reflect on whether you meet the goals you set back then. Make a comparison between what was expected, and what has been achieved. Discuss the lessons learned – for example, why some parts of the required functionality were not completed (if any), how you added some new features (and why, if any), how you would proceed with the project the same or differently in the way you do in the course, and your thoughts about the project and suggestions to Mr. Dude, etc. Please plan and include the reflections as part of your video presentation (see more details of the video presentation in the Deliverables section below).

At the conclusion of the project, your team is required to reflect thoughtfully on the decisions and progress made throughout each phase. Specifically, your reflection shall:

- Clearly revisit the goals and expectations your team set at the beginning of the semester, and discuss to what extent those goals were met.
- Provide a comparative analysis between your initial plans and the actual results achieved.
- Discuss key achievements, decisions, challenges, and lessons learned across the entire project (including both Phase 1 and Phase 2), such as:
 - Achievements you, individuals and as a team, made in this project.
 - Challenges encountered and reasons why certain required functionalities were not fully implemented (if applicable).
 - Additional features or improvements your team chose to implement, clearly explaining the motivations and outcomes.
 - Insights gained regarding how your team would approach this project differently, or similarly, in the future.
- Offer your team's overall thoughts about the project, its structure, and content, including constructive **suggestions and recommendations** to the teaching team.

Your reflection should be insightful, concise, and clearly communicate the learning outcomes and project experience. These reflections should be the content **final video presentation** (additional details regarding the presentation can be found in the **Deliverables** section).

Progress Review

To ensure steady progress toward the final implementation of your prototype, a Progress Review session will be conducted as a midpoint checkpoint on **03/23/2026**. During this review, each team is expected to demonstrate their current progress clearly and effectively.

For the Progress Review, your team should at least have implemented and be prepared to demonstrate the following functionalities:

- **Database Population:** Clearly demonstrate that your database has been successfully populated using the provided CSV files, and that integrity constraints, keys, and data consistency have been properly established.
- **User Login Functionality:** Demonstrate a secure and functional user login process, including:
 - Verification of user email and hashed passwords.
 - Role-specific login handling (Seller, Buyer, or HelpDesk).
 - Successful redirection to role-specific welcome pages upon login.

The teaching team will hold individual demo sessions (approximately 5 minutes per team) Each team will sign up for a time slot via a signup sheet. The specific schedule, signup sheet, and location details for your project demos will be provided on Canvas. The date for the Progress Review is around **03/23/2026**.

Final Deliverables

The final deliverables for Phase 2 include i) a reflective video presentation, ii) a live demonstration of your *LionAuction* system prototype, and iii) source code and documentation.

1. Video Presentation (Task 3 Reflection):

- Create a **team video presentation** (maximum of 6 minutes) focused specifically on your **reflections** from **Task 3**. Submit the video via Canvas.
- Note that this video should not be a demonstration of your implemented prototype; instead, it should critically evaluate your overall project journey. For help with making the video presentation, please refer to: <https://mediacommons.psu.edu/support/tutorials/>.

2. Live Demonstration:

- Conduct a live demonstration. During this session, your team will showcase the functionalities implemented in Task 2. Your team are required to arrive five minutes before your demonstration session and have your prototyped system started and ready to run immediately when your session starts.
- The teaching team will interactively test your web application prototype using the data provided.
- Be prepared to thoroughly explain your codes, database queries, server-side logic, and overall application architecture.

3. Source Code and Documentation Submission

- Organize all relevant project files in a project directory ready for execution in PyCharm or the IDE of your choice. The files, to be submitted in a single ZIP file, includes the following:
 - All source code files (well-documented).
 - Database setup scripts (scripts used for populating and regenerating the database from provided CSV files) and other scripts you used in the project.
 - Additional data files you created (if any).
 - A detailed README file clearly containing:
 - Brief project description/context.
 - Explanation of the directory structure and file organization.
 - Implemented features and functionalities.
 - Clear instructions for project setup, database population, and execution (including how to run your project in PyCharm Professional or your preferred IDE).
- Submit this ZIP file via Canvas.

Remember, maintaining academic integrity is essential important -- there will be no tolerance and exceptions towards violation! Even incomplete submissions are preferable over integrity violations.

Extra effort put into your project will be rewarded with extra credits.